A Project Report on

DATAGENT: Data Analytics Toolkit for Automatic Generation of Dashboards and Tailored Insights

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Engineering

in

Computer Science and Engineering - Data Science

by

Ansh Rathod(22107022)
Ayush Sharma(22107028)
Atharva Thube(22107062)
Balkrishna Yadav(22107050)

Under the Guidance of

Dr. Sameer Nanivadekar
Ms. Dipali Gat

Department of Computer Science and Engineering - Data Science
A.P. Shah Institute of Technology
G.B.Road,Kasarvadavli, Thane(W)-400615
UNIVERSITY OF MUMBAI

Academic Year 2025-2026

Approval Sheet

This Project Report entitled ***"DATAGENT: Data Analytics Toolkit for Automatic Generation of Dashboards and Tailored Insights"*** Submitted by ***"Ansh Rathod"(22107022),"Ayush Sharma"(22107028),"Atharva Thube"(22107062), "Balkrishna Yadav"(22107050)***is approved for the partial fulfillment of the requirement for the award of the degree of Ms. Dipali Gat***Bachelor of Engineering*** in ***Computer Science and Engineering - Data Science*** from ***University of Mumbai***.

Ms. Dipali Gat
Co-Guide

Dr. Sameer Nanivadekar
Guide

Dr. Pravin Adivarekar
HOD, Computer Science and Engineering - Data Science

Place:A.P.Shah Institute of Technology, Thane
Date:

CERTIFICATE

This is to certify that the project entitled ***"DATAGENT: Data Analytics Toolkit for Automatic Generation of Dashboards and Tailored Insights"*** submitted by ***"Ansh Rathod"(22107022), "Ayush Sharma" (22107028), "Atharva Thube"(22107062), "Balkrishna Yadav" (22107050)*** for the partial fulfillment of the requirement for award of a degree ***Bachelor of Engineering*** in ***Computer Science and Engineering - Data Science***,to the University of Mumbai,is a bonafide work carried out during academic year 2025-2026.

Ms. Dipali Gat
Co-Guide

Dr. Sameer Nanivadekar
Guide

Dr. Pravin Adivarekar
HOD, CSE(Data Science)

Dr. Uttam D.Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place:A.P.Shah Institute of Technology, Thane
Date:

Acknowledgement

We have great pleasure in presenting the report on **DATAGENT: Data Analytics Toolkit for Automatic Generation of Dashboards and Tailored Insights.** We take this opportunity to express our sincere thanks towards our guide **Dr. Sameer Nanivadekar** & Co-Guide **Ms. Dipali Gat** for providing the technical guidelines and suggestions regarding line of work. We would like to express our gratitude towards his constant encouragement, support and guidance through the development of project.

We thank **Dr. Pravin Adivarekar** Head of Department for his encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinators, for being encouraging throughout the course and for their guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Ansh Rathod
(22107022)

Ayush Sharma
(22107028)

Atharva Thube
(22107062)

Balkrishna Yadav
(22107050)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, We have adequately cited and referenced the original sources. We also declare that We have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(Signature)
Ansh Rathod(22107022)

(Signature)
Ayush Sharma(22107028)

(Signature)
Atharva Thube(22107062)

(Signature)
Balkrishna Yadav(22107050)

Date:

Abstract

In the current data-driven era, companies spend considerable time on data processing for gaining any meaningful insight. Activities like data cleaning, feature engineering, writing code for analysis in various programming languages, and dashboard generation are some of the tasks that involve considerable manual effort and expertise. Datagent is a smart one-stop solution for data analytics assistance. The assistant has the ability to interpret business needs in natural language and automatically generate data analysis tasks. The assistant can clean data from raw form, perform anomaly detection using Isolation Forest, create features using data libraries Pandas, Polars, and NumPy, and automatically generate scripts in Python, R, SQL, and Excel. The assistant generates code that gets automatically executed in the backend to display dynamic results and interactive dashboards in Plotly and OpenPyXL or XlsxWriter. Users can also see the data cleanup tasks in real time and download the clean data and report in PDF and Excel formats. Datagent fills the gap between data analysis experts and businesspeople using innovative technologies such as AI, NLP, and automation.

Contents

1 Introduction **1**
1.1 Motivation . . . 2
1.2 Problem Statement . . . 2
1.3 Objectives . . . 3
1.4 Scope . . . 4

2 Literature Review **6**

3 Project Design **12**
3.1 Existing System . . . 12
3.2 Proposed System . . . 13
3.2.1 Critical Components of System Architecture . . . 13
3.3 System Diagrams . . . 14
3.3.1 UML Diagram . . . 14
3.3.2 Activity Diagram . . . 15
3.3.3 Use Case Diagram . . . 17
3.3.4 Sequence Diagram . . . 17

4 Project Implementation **19**
4.1 Code Snippets . . . 19
4.2 Steps to access the System . . . 28
4.3 Timeline Sem VII . . . 30

5 Testing **33**
5.1 Software Testing . . . 33
5.1.1 Testing Objectives . . . 33
5.2 Functional Testing . . . 34

6 Result and Discussions **35**

7 Conclusion **40**

8 Future Scope **41**

Bibliography **42**

Appendices **44**

Publication **47**

List of Figures

3.1 **Critical Components of System Architecture** 14
3.2 **UML Diagram** 15
3.3 **Activity Diagram** 16
3.4 **Use Case Diagram** 17
3.5 **Sequence Diagram** 18

4.1 **Imputation Strategy Selector** 20
4.2 **4. Smart Categorical Normalization** 21
4.3 **Random Forest Feature Importance (Model-Based Importance)** . . 22
4.4 **Univariate Feature Importance (Statistical Importance)** 23
4.5 **Main** 24
4.6 **Dashboard** 24
4.7 **Dynamic R-Script Patching for Stateful Filtering** 25
4.8 **Analyze Request** 26
4.9 **Execute SQL and Generate Explainable AI insight** 27
4.10 **Generation of Excel Dashboard (Xlsxwriter)** 27
4.11 **Generation of Excel Dashboard (Openpyxl)** 28
4.12 **Generation of Interactive Excel Dashboard (Pywin32)** 28
4.13 **Gantt Chart** 32

6.1 **Data Cleaning** 35
6.2 **Data Cleaning** 36
6.3 **Feature Selection** 37
6.4 **Feature Selection** 38
6.5 **Yearly Revenue Comparison** 38
6.6 **category Sale Comparison** 39

List of Tables

2.1 Summary of Reviewed Studies 11

List of Abbreviations

IDS:	Intrusion Detection System
WSN:	Wireless Sensor Network
MANET:	Mobile Ad-Hoc Network
AODV:	Ad-Hoc On-demand Distance Vector Routing
DSR:	Dynamic Source Routing Protocol
NS2:	Network Simulator 2
ACK:	Acknowledgement
AGT:	Agent
RTR:	Router

Chapter 1

Introduction

In the modern world, data has become the foundation of decision-making across industries. Every organization collects large volumes of data daily—from customer transactions and business operations to online activities. However, transforming this raw and often unstructured data into meaningful insights is still a complex and time-consuming process. Analysts spend most of their effort on tasks like cleaning, organizing, and preparing data before meaningful analysis can even begin. This process not only delays insight generation but also demands advanced technical skills that many business users may not possess.

To address these challenges, our project "Datagent" introduces an intelligent and automated solution that simplifies the entire journey from raw data to actionable insights. Datagent acts as a smart data assistant that understands the user's requirements expressed in natural language and automatically performs the necessary steps—from cleaning and preparing the dataset to analyzing and visualizing the results.

The system provides two major modes of operation:

Data Cleaning and Preparation–where users can upload their datasets, preview them, and allow the system to automatically detect and correct inconsistencies such as missing values, duplicates, or incorrect formats. The user can observe every step of the process and view a summarized report once the cleaning is complete.

Analysis and Visualization – where users can directly start their analysis by entering questions or business requirements in simple terms. Based on these questions, the system automatically generates and executes the required analytical steps, displays results with clear explanations, and creates visual dashboards to help users interpret the findings.

Datagent supports generating insights in multiple forms, including Python-based analysis, R-based analytics, SQL query suggestions, and Excel-based dashboards. Each of these outputs is automatically generated and can be copied, edited, or downloaded by the user. The dashboards are interactive, visually appealing, and tailored to the specific analysis questions provided by the user.

Overall, Datagent brings together the power of Artificial Intelligence, Natural Language Processing, and data automation to make analytics more accessible and efficient. It removes

the barrier between technical complexity and business understanding by allowing anyone regardless of coding expertise to interact with data intuitively. Through automation, intelligent reasoning, and adaptive visualization, Datagent aims to transform how data analysis is performed, making it faster, smarter, and more user-friendly.

1.1 Motivation

In today's data-driven world, organizations rely heavily on data to make informed business decisions, forecast trends, and improve performance. However, before meaningful insights can be drawn, a large amount of time and effort goes into preparing the data-cleaning errors, handling missing values, and organizing it into a usable format. This process is often repetitive, tedious, and requires specialized technical knowledge that not every user possesses. Many individuals, especially business professionals and decision-makers, struggle to work directly with data because most analytical tools require coding skills or expertise in complex software. As a result, a large portion of potentially valuable information remains unused or delayed due to these technical barriers. Even for skilled data analysts, managing the complete process-from raw data cleaning to feature creation, analysis, and dashboard generation-consumes significant time and resources.

This challenge motivated us to develop Datagent, a system that simplifies the entire data analysis workflow. The idea was to build an intelligent assistant that could understand what the user wants to analyze, clean and prepare the dataset automatically, and generate ready-to-use analysis and dashboards-all in a simple, conversational, and interactive way. By combining automation with natural language understanding, Datagent aims to make data analytics accessible to everyone, regardless of their technical background. It removes the need to manually write code or perform repetitive steps, enabling users to focus more on interpreting results and making decisions.

The motivation behind this project is to bridge the gap between data complexity and human understanding-empowering individuals and organizations to transform raw data into actionable insights quickly and effortlessly. With Datagent, we envision a future where anyone can explore and understand their data with ease, without depending heavily on technical expertise or external tools.

1.2 Problem Statement

In the current era of digital transformation, data has become one of the most valuable assets for organizations. However, the process of turning raw data into meaningful insights is often complicated, time-consuming, and resource-intensive. Most datasets collected from different sources contain missing values, inconsistencies, errors, or irrelevant information that must be cleaned and transformed before analysis can begin. This initial stage of data preparation can take up a major portion of a data professional's time and effort.

Furthermore, performing data analysis and visualization usually requires strong technical skills in programming languages and analytical tools. Users often need to know how to

write scripts, generate SQL queries, or create dashboards manually. This makes the process difficult for non-technical business users who understand the problem but may not have the expertise to translate their questions into technical solutions. Even for skilled analysts, constantly switching between multiple tools for cleaning, coding, and dashboard creation is inefficient and repetitive. The lack of a unified system that can intelligently understand user requirements and automate these steps leads to delays, errors, and inconsistent insights.

Therefore, there is a need for an intelligent, user-friendly system that can bridge the gap between data and decision-making. Such a system should be able to understand business requirements expressed in natural language, automatically clean and prepare datasets, generate relevant analysis code, and produce clear visual dashboards—all in one place.

The problem that Datagent aims to solve is the complexity, time consumption, and technical barriers involved in the traditional data analysis workflow. By simplifying and automating this process, Datagent seeks to make data analysis more intuitive, efficient, and accessible for everyone-regardless of their technical background.

1.3 Objectives

The proposed system, DATAGENT, is designed to address the growing challenges of managing, cleaning, and analyzing large and diverse datasets by creating an AI-powered, automated data analytics platform. The main objective is to provide a comprehensive, efficient, and customizable solution that automates data preprocessing, feature selection, script generation, and dashboard creation—reducing manual effort and enabling faster, more reliable insights. By integrating advanced machine learning methods, data-cleaning libraries, and reporting tools, the system aims to make data analytics more accessible, consistent, and business-ready across industries.

- **To automate data detection, cleaning, and preprocessing**

Datagent aims to automatically detect errors, missing values, and inconsistencies within datasets of different sizes and structures. Using a high-performance data cleaning pipeline built with Polars and Pandas, along with a Rule-Based Pipeline integrated with adaptive imputation using Random Forest, Isolation Forest and KNN, it identifies the issues in the data, performs necessary cleaning steps, and prepares it for analysis, reducing the time and effort usually required for manual preprocessing.

- **To perform intelligent feature selection and creation**

The system focuses on selecting the most relevant features from the dataset and creating meaningful new measures that enhance the quality of analysis. By combining statistical techniques using NumPy, Pearson Correlation, Mutual Information, Pandas and Polars with LLM (Gemini)-driven semantic feature selection, along with analytical reasoning and AI-driven logic, Datagent ensures that the data used for visualization and reporting is both accurate and insightful.

- **Automated Script Generation**

One of the key goals is to produce ready-to-run scripts in multiple analytical languages such as Python, R, SQL, and Excel-based on the user's business requirements or analysis questions. The system interprets natural language inputs using LLM-based code generation (Gemini) and integrates Pandas and Polars to translate them into executable code snippets that can be copied, customized, and reused, allowing users to work in their preferred environment.

- **To enable interactive and automated dashboard creation**

Datagent is designed to automatically create dynamic dashboards that represent the analytical results in a clear and visual form. Whether through Python, R, or Excel, the dashboards include interactive charts, pivot tables, and slicers, built using Plotly for python and R dashboards and XlsxWriter, Openpyxl, and Pywin32 for editable and downloadable Excel dashboards, making it easy for users to explore and customize their data visually without extensive technical steps.

Together, these objectives enable DATAGENT to function as a unified and intelligent platform that automates the entire data analysis lifecycle—from raw data preprocessing to insight generation and visualization—ensuring accuracy, scalability, and efficiency across diverse analytical environments.

1.4 Scope

The scope of Datagent covers the complete journey of data handling—from raw data ingestion to final insight generation—with the help of intelligent automation and AI reasoning. The system is designed to assist users at every stage of the analytics process, making it suitable for both technical and non-technical users who want to extract value from data without extensive coding or manual effort.

Datagent provides a unified platform that brings together data cleaning, feature engineering, analysis, and dashboard creation under one intelligent system. It starts by allowing users to upload their datasets and then automatically identifies issues such as missing values, duplicates, or inconsistencies. Once the data is cleaned, it proceeds to enhance the dataset through intelligent feature selection and creation, ensuring that the information used for analysis is reliable and meaningful.

The project also extends its scope to generating analysis-ready scripts across multiple platforms — including Python, R, SQL, and Excel — based on user requirements or questions framed in natural language. These scripts are not only generated automatically but also executed to display real-time results and visual insights. An important part of Datagent's scope is the ability to create interactive dashboards that help users visualize their data effectively. These dashboards are designed to be dynamic, editable, and easy to download, offering multiple visualization options suitable for business reporting and decision-making.

Beyond analysis, the project also focuses on user experience and traceability. Every user interaction-including datasets, generated outputs, analysis steps, and results — is saved in a structured format, allowing users to review their history, maintain records, and ensure reproducibility of their work. In addition, the project is scalable and adaptable. It is designed to handle datasets of varying sizes and structures, making it suitable for applications in diverse fields such as business analytics, finance, marketing, operations, and research.

However, the project's current scope is limited to automating structured data workflows and generating analytical insights based on user-provided datasets. Real-time data streaming, deep predictive modeling, and large-scale cloud deployment are beyond the current phase but can be considered for future enhancements.

Chapter 2

Literature Review

Automated Data Processing and Feature Engineering for Deep Learning and Big Data Applications [**1**] A Survey (Journal of Information and Intelligence, 2025). In 2025, Alhassan Mumuni and Fuseini Mumuni published a survey in the Journal of Information and Intelligence examining the role of automation in data processing and feature engineering for deep learning and big data applications. The study highlights how technologies such as AutoML and Generative AI are increasingly being utilized to automate critical tasks, including data cleaning, imputation, augmentation, and feature extraction. It further reviews advanced frameworks like AutoGluon, H2O AutoML, and TPOT, which enable streamlined end-to-end machine learning pipelines. Additionally, the authors discuss the growing use of Large Language Models and Generative Adversarial Networks for intelligent data augmentation and feature creation. Despite these advancements, the study identifies key challenges such as high computational requirements, limited contextual understanding, and a lack of explainability. It emphasizes the continued importance of human-in-the-loop systems to ensure transparency, reliability, and trust in automated workflows. Drawing inspiration from these findings, the Datagent project incorporates similar automation principles for efficient data preparation while maintaining a strong focus on interpretability and user control.

Natural Language Interface for Data Visualization: Harnessing the Power of Large Language Models. [**2**] In 2024, Buchepalli Praneeth and his team presented an innovative approach in their IEEE paper titled "Natural Language Interface for Data Visualization: Harnessing the Power of Large Language Models." The study focuses on enhancing the accessibility of data analysis by allowing users to generate visualizations through natural language inputs. The proposed system leverages Large Language Models such as GPT-3.5-turbo, LLaMA 3, and Mistral 7B to interpret user queries and convert them into meaningful visual representations using Microsoft LIDA. It automates key stages of the analytics pipeline, including data summarization, goal formulation, and visualization generation, utilizing tools like Matplotlib and Seaborn. Additionally, the system demonstrates robustness by effectively handling ambiguous queries and minor input errors. The research highlights the potential of conversational AI in bridging the gap between users and complex data analytics systems. Drawing inspiration from this work, the Datagent project adopts a similar natural language-driven approach, enabling users to interact with data conversationally and generate insightful, interactive dashboards with ease.

DeepEye: Towards Automatic Data Visualization [**3**] In 2018, Yuyu Luo, Xuedi Qin, Nan Tang, and Guoliang Li introduced DeepEye at the IEEE 34th International Conference on Data Engineering, an automated data visualization system designed to simplify and optimize chart generation. The study addressed key challenges such as identifying high-quality visualizations, ranking them effectively, and selecting the most insightful options from a large search space. DeepEye integrates machine learning techniques with expert-defined rules to evaluate visualizations based on data characteristics including type, cardinality, and correlation. It intelligently recommends suitable chart types, such as bar charts for categorical data and line charts for trend analysis, while reducing redundancy through pruning strategies. The research demonstrates how automation can significantly reduce the manual effort involved in data analysis. However, it also highlights limitations such as challenges in modeling human perception and the absence of universal visualization standards. Drawing inspiration from this work, the Datagent project adopts similar intelligent ranking and recommendation mechanisms to suggest meaningful visualizations, while extending the approach with a conversational and user-friendly interface for enhanced data exploration.

AI Powered SQL Query Generator and Data Visualizer [**4**] (International Journal of Innovative Science and Research Technology). In 2024, Girish R and his team published a study titled "AI Powered SQL Query Generator and Data Visualizer" in the International Journal of Innovative Science and Research Technology. The research presents an intelligent system capable of converting natural language queries into optimized SQL statements and automatically generating visual dashboards from the resulting data. By leveraging Natural Language Processing techniques, the system interprets user intent and transforms it into structured database queries without requiring prior knowledge of SQL. It supports multiple database systems and aims to simplify data access for both technical and non-technical users. The study also emphasizes real-time analytics and AI-driven query optimization to ensure efficient and timely insights. Challenges such as handling complex queries and maintaining execution efficiency are addressed within the system design. Inspired by this research, the Datagent project incorporates a similar natural language interface for data analysis, enabling users to generate SQL queries, retrieve results, and visualize insights interactively, thereby making data exploration more seamless and accessible.

Chat2VIS: Generating Data Visualizations via Natural Language Using ChatGPT, Codex, and GPT-3. In 2023, Paula Maddigan and Teo Susnjak introduced Chat2VIS [**5**] in IEEE Access, a novel system that generates data visualizations from natural language queries using advanced Large Language Models such as GPT-3, Codex, and ChatGPT. The study demonstrates how these models can interpret user intent and automatically generate Python-based visualization code without relying on traditional NLP pipelines. By employing structured prompt engineering techniques—namely a "Description Prompt" for dataset context and a "Code Prompt" for visualization generation—the system ensures both accuracy and data privacy. It maintains security by processing metadata instead of raw data. While minor challenges such as variability in visual quality and aesthetic consistency were observed, the research establishes LLMs as an efficient and cost-effective solution for natural language-driven visualization tasks. The study signifies a shift toward conversational analytics, enabling interactive insight generation. Building upon this approach, the Datagent project

incorporates conversational feedback mechanisms, multi-turn interactions, and enhanced visualization capabilities to provide a more intuitive, secure, and intelligent data exploration experience.

AI-Powered Visual Analytics: Effortless Data Insights (International Journal for Research Trends and Innovation). [**6**] In 2025, Devi Dharshini S S, Geethanjali M, and Saran M presented their research titled "AI-Powered Visual Analytics: Effortless Data Insights" in the International Journal for Research Trends and Innovation. The study introduces an AI-driven data visualization agent that enables users to generate visualizations through natural language queries. The system leverages Large Language Models for code generation and employs a secure sandbox environment to ensure safe execution of generated scripts. Built on an intuitive Streamlit interface, it automates data preprocessing, intent understanding, and visualization generation using widely adopted charting libraries. The research highlights how AI can bridge the gap between raw data and actionable insights, making advanced analytics accessible to non-technical users. While minor challenges such as limited visualization customization and query ambiguity were observed, the system demonstrates strong potential for interactive analytics. Drawing inspiration from this work, the Datagent project extends these capabilities by incorporating conversational, multi-turn data exploration, real-time insights, and enhanced visualization flexibility to deliver a more robust and user-centric analytical experience.

Analysis of Natural Language Processing for Code Generation by Using COPRAS Method (REST Journal on Data Analytics and Artificial Intelligence). In 2024, Madhusudhan Dasari Sreeramulu published a study titled "Analysis of Natural Language Processing for Code Generation by Using COPRAS Method" [**7**] in the REST Journal on Data Analytics and Artificial Intelligence. The research presents an NLP-based framework that converts natural language instructions into executable code, aiming to bridge the gap between non-technical users and software development processes. Using the COPRAS method, the study evaluates five deep learning models—Seq2Seq, SNM, Tree2Tree, TRANX, and Coarse-to-Fine—across multiple criteria including efficiency, readability, cost, and usability. Among these, TRANX is identified as the most effective model due to its superior performance in efficiency and code readability. The framework combines deep learning with decision-analysis techniques to ensure a transparent and structured evaluation process. Despite certain limitations such as dependence on quantitative metrics and challenges in handling ambiguous language inputs, the study highlights the transformative potential of AI-driven code generation. Inspired by this work, the Datagent project adopts a similar approach by leveraging AI and NLP to interpret user inputs and automate intelligent data processing and analytical tasks.

[8] Leveraging Large Language Models for Data Analysis Automation (PLOS ONE, 2025). In 2025, Jacqueline A. Jansen, Artür Manukyan, Nour Al Khoury, and Altuna Akalin introduced "Leveraging Large Language Models for Data Analysis Automation" in PLOS ONE. The study presents mergen, an R package designed to automate data analysis workflows using Large Language Models. The system translates natural language descriptions of analytical tasks into executable R code, with a primary focus on bioinformatics and omics data analysis. It utilizes advanced prompting strategies and self-correction mechanisms to improve

code accuracy and reliability. The research compares models such as GPT-3.5-turbo and GPT-4 across tasks of varying complexity, demonstrating that while simpler tasks are handled efficiently, more complex workflows still require expert supervision. Despite challenges related to scalability and occasional inaccuracies in generated code, the study highlights the growing potential of LLMs in accelerating routine data analysis processes. Drawing inspiration from this research, the Datagent project incorporates a similar LLM-driven framework for automating analytical pipelines, enhanced with multi-layered validation and contextual fine-tuning to ensure higher accuracy and adaptability across diverse use cases.

Plotplay: An Automated Data Visualization Website Using Python and Plotly. [9] In 2023, Zarinabegam Kasimali Mundargi and her team introduced Plotplay in an IEEE publication, presenting a web-based platform aimed at simplifying data visualization for users without programming expertise. Built using Python and Plotly, the system enables users to upload datasets in CSV format and generate various types of visualizations, including scatter plots, histograms, and correlation matrices. Its user-friendly interface guides users through data upload, visualization selection, and parameter customization, producing interactive charts with features such as zooming, panning, and exporting. The study demonstrates how Plotplay enhances accessibility and engagement in data exploration, particularly for beginners and students. However, it identifies limitations such as restricted customization options and the absence of automated data preprocessing capabilities. The authors suggest future enhancements including outlier detection and data cleaning features. Inspired by this work, the Datagent project builds upon these concepts by integrating intelligent visualization recommendations, automated preprocessing, and collaborative functionalities, thereby offering a more comprehensive and insightful data exploration platform.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
1	*Automated Data Processing and Feature Engineering for Deep Learning and Big Data Applications: A Survey*	Alhassan Mumuni, Fuseini Mumuni	2025	Survey of automated preprocessing, augmentation, feature engineering, AutoML methods (GANs, VAEs, Auto-Sklearn).	Limited automation, scalability issues, poor generalization, lack of explainability, task support limitations.
2	*Natural Language Interface for Data Visualization*	Buchepalli Praneeth, Mohana, Ashutosh Kumar Singh, Suma B, Jyothika K Raju	2024	Visualization tool using LLMs (Mistral 7B, LLaMa3 8B, GPT-3.5-turbo) + LIDA for data loading, summarization, goal generation, visualization.	Ambiguity in queries, dependence on LLMs, no real-time data, limited customization, weak robustness, privacy concerns.
3	*DeepEye: Towards Automatic Data Visualization*	Yuyu Luo, Xuedi Qin, Nan Tang, Guoliang Li	2018	ML models (Decision Tree, LambdaMART) + expert rules for visualization selection.	Dependent on training data, limited chart types, scalability issues, no NL interaction.
4	*AI Powered SQL Query Generator and Data Visualizer*	Girish R., Swasthik K., Veera Mani Kandan S., Muralidharan S., Vinoth M., Surendar Prakash T.	2024	NLP-based SQL query generation, AI-optimized queries, MySQL/PostgreSQL integration, auto dashboards.	Conceptual MVP, lacks scalability, basic visualizations, no enterprise integration, monolingual NLP.
5	*Chat2VIS: Generating Data Visualizations via NL Using ChatGPT, Codex, GPT-3*	Paula Maddigan, Teo Susnjak	2023	Chat2VIS system using GPT-3, Codex, ChatGPT + prompt engineering to generate visualizations.	Aesthetic control issues, variability in outputs, limited evaluation, lacks benchmarks.
6	*AI-Powered Visual Analytics: Effortless Data Insights*	Devi Dharshini S. S., Geethanjali M., Saran M.	2025	Together AI LLMs + Pandas, Plotly, Seaborn, Matplotlib + Streamlit for NL queries to real-time visualization.	Ambiguous query handling, limited customization, weak error handling, lacks voice input & dashboarding.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
7	*Analysis of NLP for Code Generation using COPRAS*	Madhusudhan Dasari, Sreeramulu	2024	COPRAS applied to Seq2Seq, SNM, Tree2Tree, TRANX, Coarse-to-Fine ranking.	No data transparency, oversimplified metrics, lacks domain-specific context.
8	*Leveraging LLMs for Data Analysis Automation*	Jacqueline A. Jansen, Artur Manukyan, Nour Al Khoury, Altuna Akalin	2025	R package `mergen` + GPT-3.5/GPT-4, prompt engineering, self-correction, applied to bioinformatics tasks.	Executable code not always correct, poor on complex tasks, privacy issues.
9	*Plotplay: Automated Data Visualization Website using Python and Plotly*	Zarinabegam K. Mundargi, Rohan More, Krisha Patel, Sudhanshu Pathrabe, Aayushi Patel, Shreyash Patil	2023	Flask-based tool with Pandas + Plotly (10 chart types).	CSV-only, no preprocessing, limited plots, no customization.

Table 2.1: Summary of Reviewed Studies

Chapter 3

Project Design

Our project, Datagent, is an intelligent AI-powered system designed to simplify complex data and coding tasks through natural language interaction. It enables users — even those without technical expertise — to describe their analytical or coding needs in plain English and instantly receive accurate, executable results. By combining Natural Language Processing (NLP) with Artificial Intelligence, automated code generation and data analysis, Datagent bridges the gap between human intent and machine execution. The system focuses on enhancing accessibility, efficiency, and reliability in data-driven workflows, empowering individuals and organizations to transform raw data into actionable insights with minimal effort.

3.1 Existing System

Existing systems in data analytics lack end-to-end automation, as they typically focus on isolated stages such as preprocessing, visualization, or query generation rather than providing a complete pipeline from raw data to insights. This forces users to manually integrate multiple tools, increasing complexity and effort. Additionally, data preprocessing in these systems is often inefficient and non-adaptive, offering only basic cleaning techniques without intelligent error detection or advanced imputation methods, making them less effective for handling diverse and complex datasets.

Furthermore, feature selection and creation are generally limited to statistical or rule-based approaches, lacking semantic understanding, which results in less meaningful and lower-quality insights. Code generation capabilities, although supported by some NLP and LLM-based systems, are often restricted to specific tasks or languages and struggle with complex workflows, requiring manual correction and reducing reliability.

Moreover, these systems are highly fragmented, requiring users to depend on multiple platforms for different stages of analysis, which reduces efficiency and creates integration challenges. They also face scalability and performance constraints, as many require high computational resources and are not optimized for handling large-scale or real-time data effectively.

3.2 Proposed System

The proposed system, DATAGENT, is an AI-powered data analytics platform designed to automate the complete data analysis lifecycle, including data preprocessing, feature selection, script generation, and dashboard creation. It provides a comprehensive and efficient solution for handling large and diverse datasets, reducing manual effort and enabling faster, reliable insights.

DATAGENT includes an intelligent preprocessing module that automatically detects errors, missing values, and inconsistencies using advanced techniques such as Random Forest, Isolation Forest, and KNN, ensuring accurate and adaptive data cleaning. It also performs smart feature selection and creation by combining statistical methods with LLM-driven semantic understanding, leading to more meaningful insights.

The system supports automated multi-language script generation (Python, R, SQL, Excel) from natural language inputs, making it flexible and user-friendly. Additionally, it generates interactive and customizable dashboards using tools like Plotly and Excel libraries, enabling easy data exploration.

Overall, DATAGENT overcomes the limitations of existing systems by providing a unified, end-to-end platform with better automation, adaptability, scalability, and user accessibility.

3.2.1 Critical Components of System Architecture

The system architecture depicted in the Image is structured into two primary domains: the Client Side and the Server Side, each playing a distinct role in the data analysis workflow. On the Client Side, users interact through a comprehensive Interface that supports data transformation, cleaning, feature selection, and analysis using Python, R, and SQL. This interface also includes dashboarding capabilities across multiple platforms, including Excel, Python, and R. Accessibility features mirror these functionalities, ensuring users can perform data cleaning, feature engineering, and analysis seamlessly across different tools.

On the server side, the architecture handles the core data processing and model logic. Data cleaning is performed using libraries such as Pandas and Polars, along with Isolation Forest for anomaly detection and Sentence Transformers for semantic text processing. Feature selection and creation are implemented using Pandas, NumPy, and a custom framework (AIFILL), combining statistical and intelligent approaches. For code generation and execution, the system leverages a Large Language Model (Gemini) integrated with data processing libraries. Visualization is handled using Plotly for interactive dashboards, while Excel-related operations are managed using XlsxWriter, OpenPyXL, and Pywin32, enabling dynamic generation of downloadable and editable Excel dashboards.

The architecture is unified by a set of connecting elements: business insights, analysis questions, and datasets feed into both client and server components. User inputs are routed through an API Gateway to the server, where processing occurs, and outputs are generated and returned. This modular and tool-rich design ensures scalability, flexibility, and robust

analytical capabilities across the entire data pipeline.

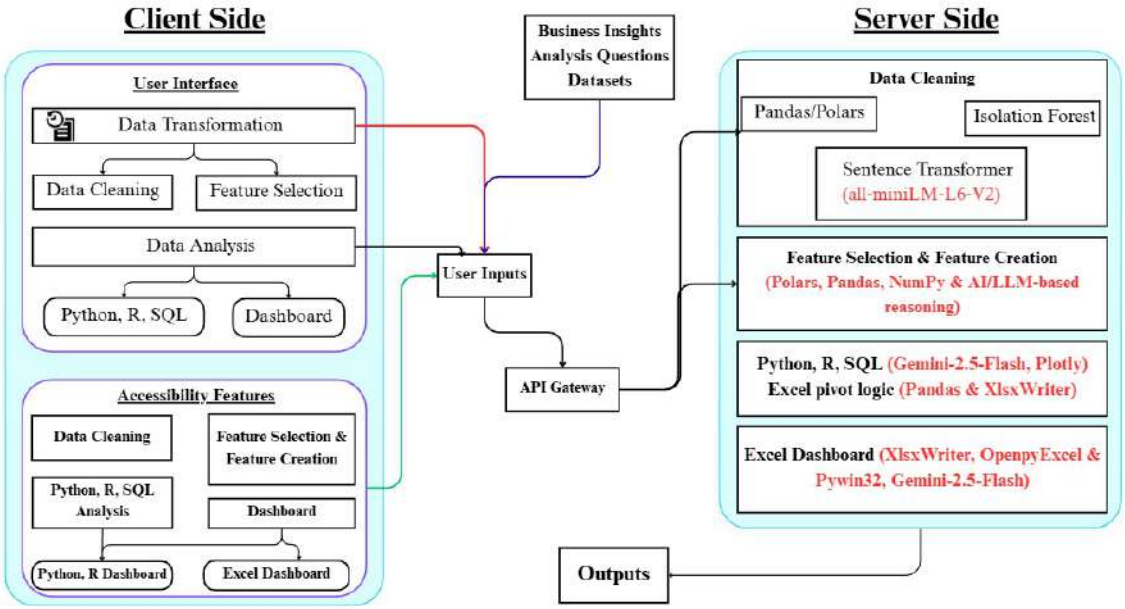

Figure 3.1: **Critical Components of System Architecture**

The overall architecture of the proposed system is illustrated in Figure 3.1. It demonstrates how various modules interact to process input datasets, perform automated analysis, and generate outputs such as cleaned data, analytical scripts, and fully interactive Excel dashboards tailored to user requirements.

3.3 System Diagrams

To provide a clear understanding of the internal structure and functionality of the proposed system, various UML (Unified Modeling Language) diagrams have been designed. These diagrams visually represent how different components of the system interact, the flow of data, and user-system relationships. The section includes the Activity Diagram, Use Case Diagram, and Sequence Diagram, each serving a distinct purpose in illustrating system behavior and workflow. Together, these diagrams form the blueprint of the system architecture, helping developers and stakeholders visualize the logical flow, process sequence, and interactions between different modules of the Datagent platform.

3.3.1 UML Diagram

UML (Unified Modeling Language) is a standardized visual language used to model, design, and document the structure and behavior of software systems. It helps developers, analysts, and stakeholders understand the system architecture and interactions before implementation. UML diagrams provide a clear representation of how different components, processes, and users interact within a system. They are broadly classified into structural diagrams

(which show the static aspects like classes, objects, and system architecture) and behavioral diagrams (which depict the dynamic aspects like workflows, activities, and interactions).

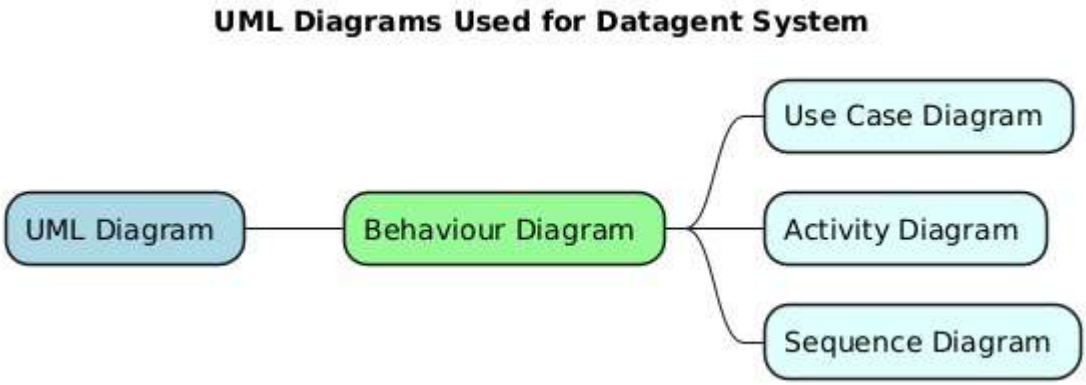

Figure 3.2: **UML Diagram**

In the above figure 3.2, the system structure is represented using behavioral UML diagrams, including the Activity Diagram, User Diagram, and Sequence Diagram. These diagrams illustrate how different components of the Datagent system interact dynamically — depicting workflows, user interactions, and the sequence of operations.

3.3.2 Activity Diagram

An Activity Diagram represents the dynamic flow of control or data within a system. It visually outlines the sequence of activities, decision points, and parallel processes, making it ideal for modeling workflows such as data preprocessing, analysis, and report generation in a project like Datagent.

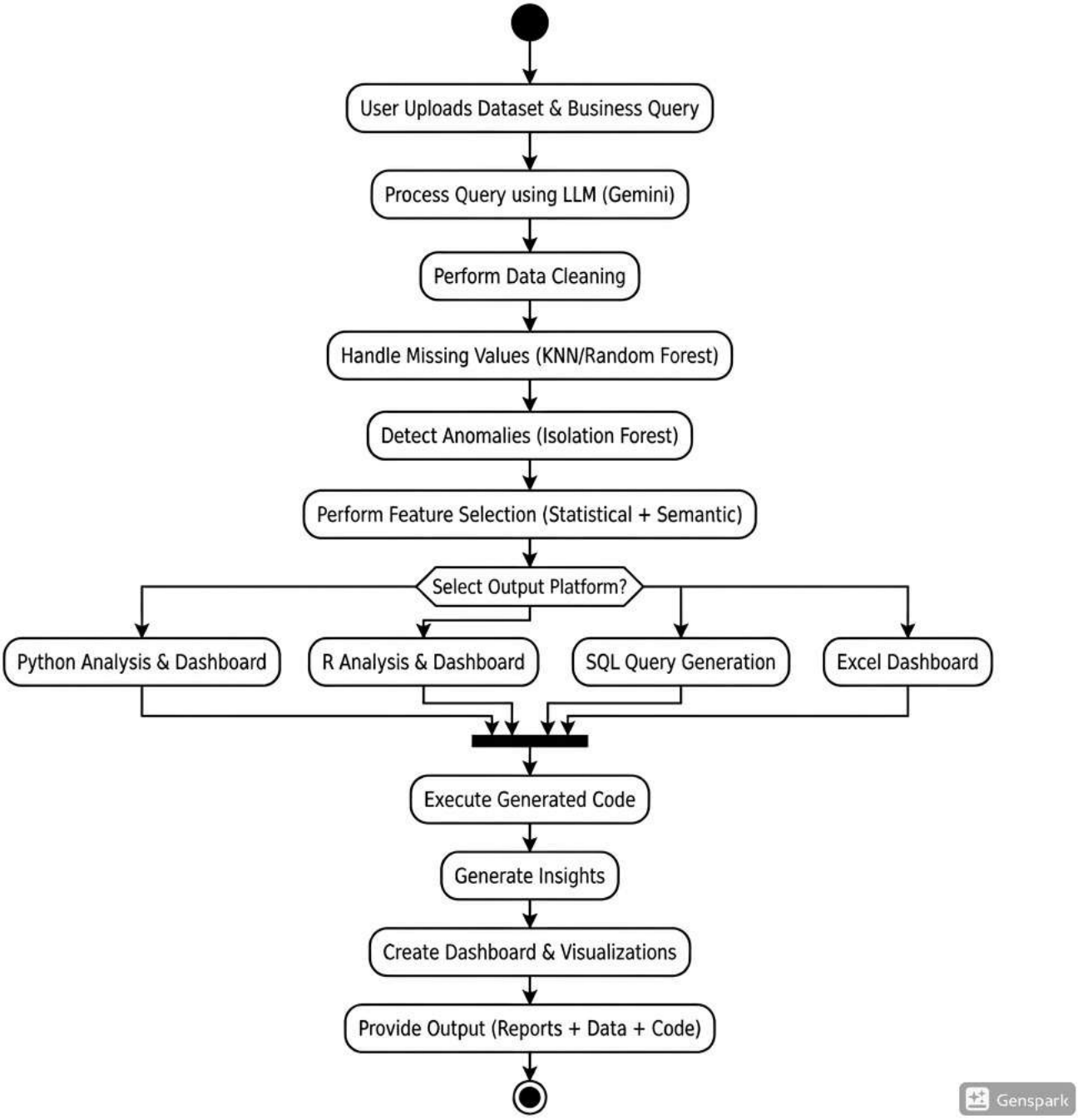

Figure 3.3: **Activity Diagram**

The above Figure 3.3 illustrates the step-by-step workflow of the system — starting from user login and dataset upload to data cleaning, feature engineering, analysis, and dashboard generation. It captures how each process flows sequentially, showing decision points such as whether the user chooses data cleaning or direct analysis, and how the system responds at each stage to ensure smooth and automated data processing.

3.3.3 Use Case Diagram

A Use Case Diagram visually represents the interaction between users (actors) and the system, showcasing the various functionalities the system provides. It helps identify what the user can do and how the system responds to those actions, forming the foundation for understanding system requirements and user behavior.

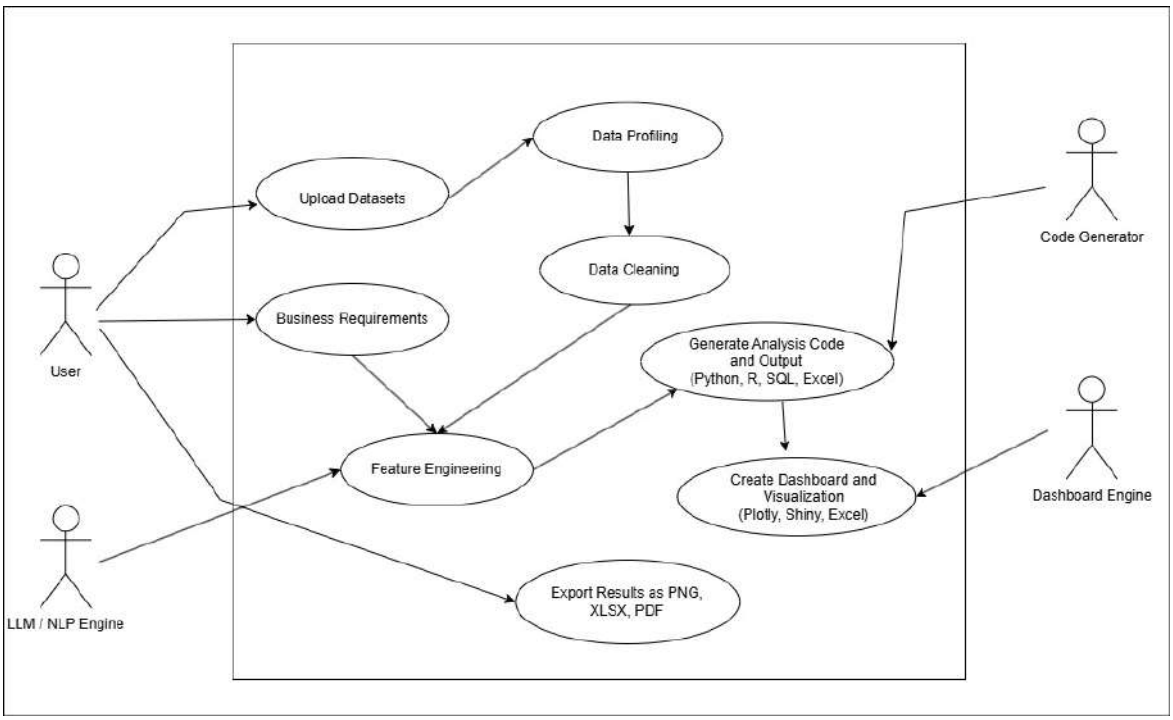

Figure 3.4: **Use Case Diagram**

The above Figure 3.4 illustrates the interaction between the user and the Datagent system. It represents key functionalities such as data upload, data cleaning, feature engineering, analysis execution using Python, R, SQL, or Excel, and dashboard generation. The diagram highlights how the user initiates each process, while the system — powered by LLMs and automation modules — performs tasks like data validation, code generation, and visualization creation.

3.3.4 Sequence Diagram

A Sequence Diagram is a type of UML behavioral diagram that depicts how objects or system components interact with each other over time. It focuses on the flow of messages, function calls, and responses between different parts of the system, showing the order and timing of interactions. This helps visualize the dynamic behavior of a system and understand how processes collaborate to complete a task.

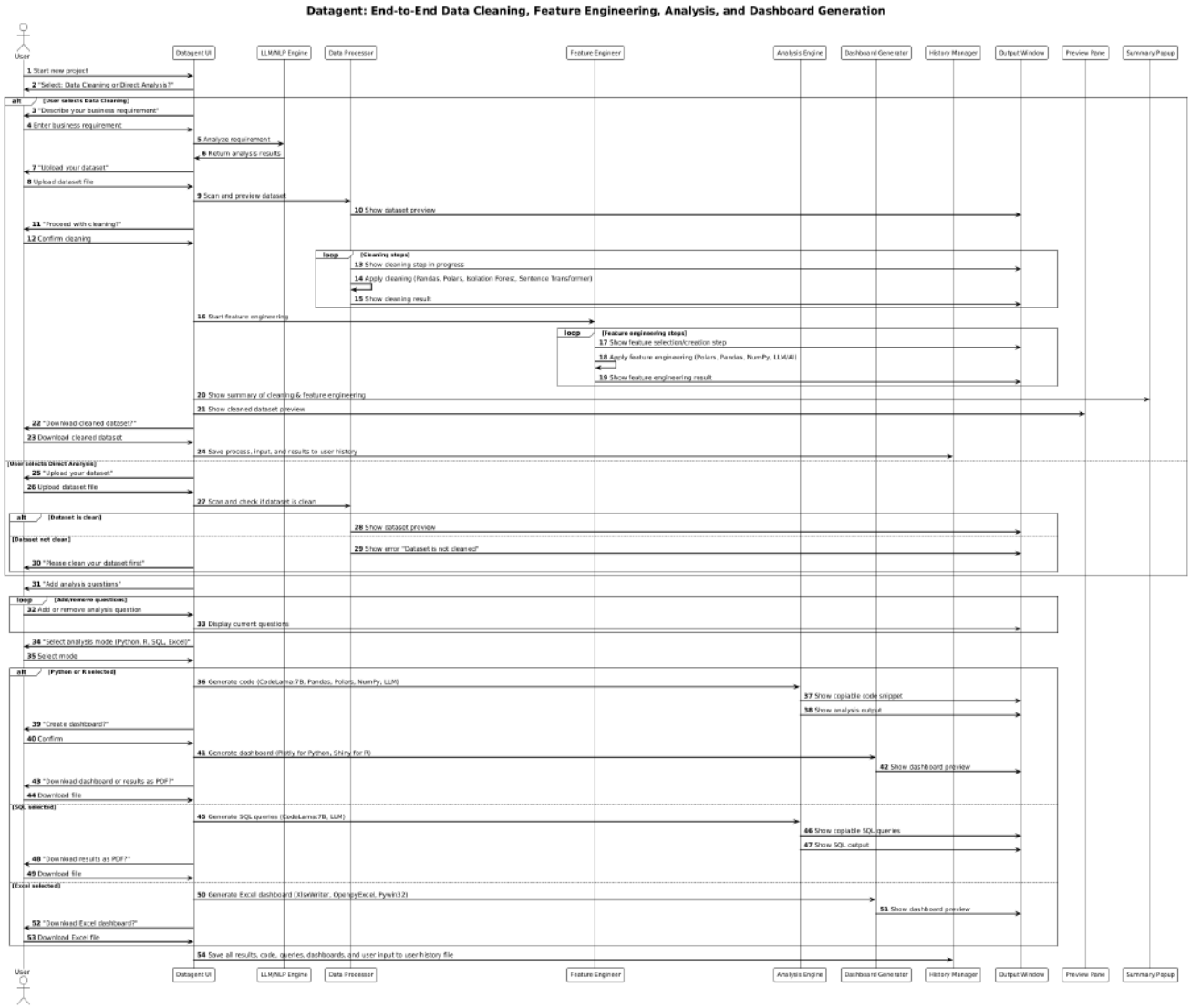

Figure 3.5: **Sequence Diagram**

The above Figure 3.5 illustrates the dynamic interaction between the user, the Flask backend, the AI models (Google Gemini), and the database modules. It shows how the system processes a user's actions step by step — starting from entering business requirements or uploading a dataset, followed by the AI-driven cleaning, feature engineering, code generation, and finally the dashboard creation. This diagram helps in understanding the real-time flow of control and data across different components.

Chapter 4

Project Implementation

This chapter presents the core aspects of the implementation, including essential feature specific code snippets and the overall project timeline. It illustrates how the system modules were developed, how different components were integrated, and how the AI models were applied to real-time data processing. The timeline highlights the structured progression of development, while the code snippets emphasize technically critical parts of the implementation.

4.1 Code Snippets

The following section presents key implementation components of the Datagent system, showcasing essential code snippets and execution workflows that enable the platform's intelligent automation. Each code segment represents a critical stage in the system's operation — including data ingestion, cleaning, feature engineering, LLM-based analysis generation, and dashboard creation

```
backend > datagent.py > ImputationStrategySelector > select_strategy
class ImputationStrategySelector:
    def select_strategy(self, df: pd.DataFrame, col: str, null_pct: float,
    def select_strategy(self, df: pd.DataFrame, col: str, null_pct: float,
                        semantic_type: str, relationships: List[Dict]) -> Tuple[str, float, str]:
        """
        🧠 DECISION LOGIC

        Returns: (strategy, confidence, reason)
        """

        missing_mask = df[col].isna()
        n_missing = missing_mask.sum()
        n_available = (~missing_mask).sum()

        # ============================================================
        # RULE 1: Too sparse → skip
        # ============================================================
        if null_pct > Config.MAX_NULL_PERCENT:
            self.strategy_stats['skipped'] += 1
            return 'skip', 0.2, f'Too sparse ({null_pct:.1f}% missing)'


        IDENTITY_SEMANTIC_TYPES = {'TransactionID', 'CustomerID', 'ProductID'}
        IDENTITY_KEYWORDS = ('id', 'phone', 'mobile', 'email', 'mail',
                             'name', 'address', 'url', 'uuid', 'guid')
        col_lower_check = col.lower().replace(' ', '_')
        is_identity_col = (
            semantic_type in IDENTITY_SEMANTIC_TYPES
            or any(kw in col_lower_check for kw in IDENTITY_KEYWORDS)
        )
        if is_identity_col:
            self.strategy_stats['skipped'] += 1
```

Figure 4.1: **Imputation Strategy Selector**

In Figure 4.1, This component intelligently determines the most appropriate method for handling missing values by analyzing factors such as the percentage of missing data, data type, and availability of related features. It dynamically selects between advanced machine learning techniques like KNN and Random Forest, or simpler statistical methods like mean, median, and mode. This adaptive approach ensures higher accuracy, prevents overfitting, and maintains data integrity across different scenarios.

```python
class SmartCategoricalNormalizer:

        if n_unique <= MAX_UNIQUE_FOR_FUZZY:
            # ---------------------------------------------------
            # Step 1 — Build fuzzy similarity groups
            # ---------------------------------------------------
            groups = defaultdict(list)

            for value in base.unique():
                placed = False
                for group_key in list(groups.keys()):
                    similarity = max(
                        fuzz.ratio(value, group_key),
                        fuzz.partial_ratio(value, group_key),
                        fuzz.token_sort_ratio(value, group_key),
                        fuzz.token_set_ratio(value, group_key)
                    ) / 100

                    value_words = set(value.split())
                    key_words   = set(group_key.split())
                    overlap     = value_words.intersection(key_words)


                    # ———————————————————————————————————————————————
                    both_single_word = (len(value_words) == 1 and len(key_words) == 1)
                    passes_overlap   = bool(overlap) or both_single_word

                    if similarity >= 0.82 and passes_overlap:
                        groups[group_key].append(value)
                        placed = True
                        break

                if not placed:
```

Figure 4.2: **4. Smart Categorical Normalization**

In Figure 4.2, This method improves data consistency by identifying and correcting variations in categorical values using fuzzy matching techniques. It detects similar values with minor differences (such as spelling errors, abbreviations, or formatting issues) and maps them to a standard form. By reducing redundancy and inconsistency in categorical data, it enhances data quality and ensures more reliable analysis and visualization.

```python
def _rf_importance(df_pd: pd.DataFrame,
                   all_cols: List[str],
                   target_col: str) -> Dict[str, float]:
    """
    Train a RandomForestRegressor with the primary numeric target and return
    normalised feature importance scores for all input columns.

    Categorical columns are label-encoded.
    Date columns are excluded (they are always kept by domain rule).
    Returns an empty dict if no suitable target or insufficient data.
    """
    if not target_col or target_col not in df_pd.columns or len(df_pd) < 50:
        return {}

    y = pd.to_numeric(df_pd[target_col].astype(str).str.replace(r"[₹$£€,]", "", regex=True),
                      errors="coerce").fillna(0)

    feature_cols = [c for c in all_cols if c != target_col]
    X_parts: List[pd.Series] = []
    used_cols: List[str] = []

    for col in feature_cols:
        col_s = df_pd[col].astype(str).str.replace(r"[₹$£€,]", "", regex=True)
        num   = pd.to_numeric(col_s, errors="coerce")

        if num.notna().mean() > 0.75:
            X_parts.append(num.fillna(num.median()))
            used_cols.append(col)
        else:
            # Encode categorical
            try:
                le  = LabelEncoder()
```

Figure 4.3: **Random Forest Feature Importance (Model-Based Importance)**

In Figure 4.3, This function determines feature importance using a Random Forest model, which consists of multiple decision trees. Each feature's importance is calculated based on how much it contributes to reducing prediction error across all trees. Unlike statistical methods, it captures complex non-linear relationships and interactions between features. The model automatically handles both numerical and encoded categorical data during training. The final importance scores are normalized, making it easy to compare features and identify the most influential ones in predicting the target variable.

```python
def _univariate_scores(df_pd: pd.DataFrame,
                       all_cols: List[str],
                       target_col: str) -> Dict[str, float]:
    """
    Compute a normalised [0,1] univariate relevance score for every column
    against the primary numeric target (revenue/total/profit).

    Numeric columns  → Pearson |r| + Mutual Information (average of both).
    Categorical cols → Mutual Information classif (target binned into quartiles).
    Date columns     → skipped (always kept via domain rule).
    """
    if not target_col or target_col not in df_pd.columns:
        return {c: 0.5 for c in all_cols}  # neutral if no target

    y = pd.to_numeric(df_pd[target_col].astype(str).str.replace(r"[₹$£€,]", "", regex=True),
                      errors="coerce").fillna(0)

    # Bin target into 4 classes for MI classification
    y_binned = pd.qcut(y, q=4, labels=False, duplicates="drop")

    scores: Dict[str, float] = {}

    for col in all_cols:
        if col == target_col:
            scores[col] = 1.0
            continue

        col_s = df_pd[col].astype(str).str.replace(r"[₹$£€,]", "", regex=True)
        col_num = pd.to_numeric(col_s, errors="coerce")

        try:
            if col_num.notna().mean() > 0.75:
```

Figure 4.4: **Univariate Feature Importance (Statistical Importance)**

In the above figure 4.4,This function evaluates the relevance of each feature by analyzing its statistical relationship with the target variable using methods like Pearson Correlation and Mutual Information. Pearson correlation measures the strength of linear relationships, while mutual information captures both linear and non-linear dependencies. The function handles both numerical and categorical features by converting them into suitable formats for analysis. By combining these metrics, it provides a balanced and reliable importance score for each feature. This helps in identifying which variables have the strongest influence on the target before applying machine learning models.

```python
# — POST-EXEC: Convert matplotlib figures → Plotly (100% match guaranteed) —
try:
    _mpl_charts: List[Dict[str, Any]] = []
    _fig_nums = plt.get_fignums()
    for _i, _fig_num in enumerate(_fig_nums):
        try:
            _mpl_fig = plt.figure(_fig_num)
            try: _mpl_fig.canvas.draw()
            except Exception: pass

            _axes = _mpl_fig.get_axes()
            if not _axes: continue
            _ax = _axes[0]

            _title = _ax.get_title() or f"Chart {_i + 1}"
            _xlabel = _ax.get_xlabel() or ""
            _ylabel = _ax.get_ylabel() or ""

            _lines = _ax.get_lines()
            _patches = _ax.patches
            _plotly_fig = go.Figure()
            has_data = False

            # — LINE CHART —
            if _lines and not _patches:
                _line = _lines[0]
                _xdata = list(_line.get_xdata())
                _ydata = list(_line.get_ydata())
                _x_formatted = []
                is_date_axis = ("Date" in type(_ax.xaxis.get_major_formatter()).__name__ or "Date" in type(_ax.xaxis.get_major_locator()).__name__)
                for x in _xdata:
                    if isinstance(x, pd.Timestamp): _x_formatted.append(x.strftime('%Y-%m-%d'))
                    elif is_date_axis and isinstance(x, Number):
                        try: _x_formatted.append(mdates.num2date(x).strftime('%Y-%m-%d'))
                        except Exception: _x_formatted.append(str(x))
                    else: _x_formatted.append(str(x))
```

Figure 4.5: **Main**

In the above Figure 4.5 workflow, the system seamlessly bridges static data visualization with interactive dashboarding. The backend algorithmic engine intercepts static Matplotlib figures generated by the AI, programmatically extracting the underlying data points, axis labels, and graphical structures like lines and bar patches. It then automatically reconstructs these extracted elements into highly interactive Plotly JSON objects for frontend rendering.

```python
def rerun_visuals(df_filtered):
    """
    Re-executes the last_analysis.py code using the FILTERED dataset.
    Writes output to a temp dir to avoid corrupting the original files.
    Returns list of Plotly chart dicts (title + plotly_json) on success, [] on failure.
    """
    import tempfile, glob as _glob

    # Prefer code from disk (most up-to-date) over session state
    code = None
    if LAST_ANALYSIS_PATH.exists():
        try:
            with open(LAST_ANALYSIS_PATH, 'r', encoding='utf-8') as f:
                code = f.read()
        except Exception:
            pass
    if not code:
        code = st.session_state.get("last_analysis_code", "")
    if not code:
        return []

    charts = []
    with tempfile.TemporaryDirectory() as tmp_dir:
        tmp_path = Path(tmp_dir)

        local_vars = {
            'df': df_filtered,
            'pd': pd,
            'plt': plt,
            'sns': sns,
            'go': go,
            'px': px,
            'make_subplots': make_subplots,
            'json': json,
            'Path': Path,
            'FuncFormatter': FuncFormatter,
            're': re,
            'os': os,
            'math': math,
            'currency_fmt': FuncFormatter(currency_fmt_func),
            'TEMP_CHARTS_DIR': tmp_path,  # redirect to temp dir
            'glob': _glob,
        }
```

Figure 4.6: **Dashboard**

In the above Figure 4.6, workflow, the dashboard engine enables dynamic, stateful data exploration when a user applies slicers and filters. It achieves this by retrieving the previously cached, AI-generated Python analysis code and dynamically re-executing it against the newly filtered dataset within an isolated temporary environment. This dual mechanism ensures that users receive instantly updated visual insights without incurring additional latency or requiring redundant script generation from the Large Language Model.

```python
filtered_csv_path = str((TEMP_DATA_DIR / "filtered_dashboard.csv").resolve()).replace("\\", "/")
df_filtered.to_csv(filtered_csv_path, index=False)

# Use persistent folder instead of tempfile.TemporaryDirectory
filter_out_dir = (TEMP_CHARTS_DIR / "filtered_run").resolve()
filter_out_dir.mkdir(parents=True, exist_ok=True)
# Clear previous run files
for old_f in filter_out_dir.glob("*"):
    try:
        old_f.unlink()
    except Exception:
        pass
filter_out_str = str(filter_out_dir).replace("\\", "/")

# Patch 1: read.csv → filtered CSV
patched_code = re.sub(
    r"read\.csv\s*\(\s*['\"].*?['\"]",
    f"read.csv('{filtered_csv_path}'",
    r_code, count=1, flags=re.IGNORECASE
)

# Patch 2: CHARTS_DIR → persistent filtered_run dir
patched_code = re.sub(
    r"CHARTS_DIR\s*<-\s*['\"][^\'\"].*?['\"]",
    f"CHARTS_DIR <- '{filter_out_str}'",
    patched_code
)

r_filter_path = str((Path(".") / "temp_analysis_filtered.R").resolve())
_run_rscript(patched_code, r_filter_path)
```

Figure 4.7: **Dynamic R-Script Patching for Stateful Filtering**

In the above figure 4.7, workflow, the system enables stateful dashboard interactivity without requiring redundant calls to the Large Language Model. The backend engine intercepts the previously AI-generated R script and programmatically updates its execution parameters using regular expressions. It dynamically patches the script to ingest a newly filtered dataset and redirects the output to an isolated directory, immediately executing this modified code to rapidly regenerate targeted, interactive visualizations.

```python
@app.post("/api/analyze")
async def analyze_data(req: AnalyzeRequest):
    """
    1. Generates highly optimized SQL queries based on user questions.
    2. Executes them one by one.
    3. Runs Explainable AI on the results of each query.
    """
    if not api_key:
        raise HTTPException(status_code=500, detail="Google API Key is missing on the backend.")

    # 1. Ask AI to generate optimized SQL for all questions
    prompt_sql = f"""
    I am querying a MySQL database for the RETAIL SALES domain.
    Schema: {req.schema_json}

    RETAIL BUSINESS CONTEXT:
    - "Revenue" or "Sales" = total amount or (price * quantity).
    - "Best Sellers" = sort by highest revenue or highest quantity.
    - Write HIGHLY EFFICIENT, optimized MySQL queries. Use AS aliases for readable columns.
    - Always use proper aggregations (SUM, COUNT) and GROUP BY clauses where needed.

    User Question(s): "{req.question}"

    TASK: Break the user's input into individual questions. Generate an optimized SQL query for each.
    Return ONLY a raw JSON array. No markdown, no backticks.
    Format: [{"question": "text of question", "sql": "SELECT ..."}]
    """

    try:
        resp_sql = model.generate_content(prompt_sql)
        clean_text = resp_sql.text.replace("```json", "").replace("```", "").strip()
        queries = json.loads(clean_text)
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Failed to parse questions into SQL: {str(e)}\n\nAI Response: {resp_sql.text if 'resp_sql' in locals() else 'None'}")

    results = []
    conn = get_db_connection(req.creds)
```

Figure 4.8: **Analyze Request**

In the above figure 4.8 workflow, the system converts user queries into optimized SQL statements using a Large Language Model. It interprets the database schema and business context to break down the input into structured sub-queries. Each query is designed with proper aggregations and efficiency, ensuring accurate and meaningful data extraction from the database.

```python
# 2. Execute SQL & Generate Explainable AI insight ONE BY ONE
try:
    for item in queries:
        q_text = item.get("question", req.question)
        sql = item.get("sql", "")

        try:
            # Run the optimized SQL query
            df = pd.read_sql(sql, conn)

            # Convert DataFrame to a list of dictionaries for the Next.js frontend
            data_records = df.to_dict(orient="records")
            columns = list(df.columns)
            error = None

            # 3. Explainable AI: Generate insight based on the table output
            if len(df) == 0:
                explanation = "No data found for this specific query."
            else:
                # Send only the first 5 rows to AI so we don't exceed token limits
                if len(df) <= 5:
                    data_str = df.to_string(index=False)
                else:
                    data_str = f"First 5 rows:\n{df.head(5).to_string(index=False)}\n(Total rows: {len(df)})"

                prompt_insight = f"""
                You are an expert Retail Data Analyst.
                Question: "{q_text}"
                SQL Executed: "{sql}"

                Data Results:
                {data_str}

                Provide a VERY SHORT, 1-2 sentence business explanation of these results.
                Do NOT explain how the SQL works. Just give the direct business insight.
                """
                resp_insight = model.generate_content(prompt_insight)
                explanation = resp_insight.text.strip()

        except Exception as e:
            data_records, columns, explanation = [], [], None
            error = str(e)
```

Figure 4.9: **Execute SQL and Generate Explainable AI insight**

In the above figure 4.9 workflow, the system executes each generated SQL query sequentially and processes the results. If data is available, it applies Explainable AI to generate concise business insights based on the output. This step ensures that users receive clear, actionable interpretations of the data, simplifying analysis and supporting effective decision-making.

```python
try:
    with pd.ExcelWriter(file_path, engine='xlsxwriter') as writer:
        workbook = writer.book

        current_session["raw_df"].to_excel(writer, sheet_name='Raw_Dataset', index=False)

        current_session["df"].to_excel(writer, sheet_name='Cleaned_Dataset', index=False)
        ws_clean = writer.sheets['Cleaned_Dataset']
        max_row, max_col = current_session["df"].shape
        ws_clean.add_table(0, 0, max_row, max_col - 1, {
            'columns': [{'header': c} for c in current_session["df"].columns],
            'name': 'MainTable',
            'style': 'TableStyleMedium9'
        })

        if current_session["analysis_plan"]:
            insight_cols = ['question', 'chart_type', 'x_axis', 'y_axis', 'aggregation', 'slicer_column']
            plan_df = pd.DataFrame(current_session["analysis_plan"])
            available = [c for c in insight_cols if c in plan_df.columns]
            plan_df[available].to_excel(writer, sheet_name='AI_Insights', index=False)
```

Figure 4.10: **Generation of Excel Dashboard (Xlsxwriter)**

In the above figure 4.10 code, the system illustrates the generation of a static dashboard or visualization in Excel for the provided analytical question. This is achieved using the XlsxWriter library, which enables the creation of static Excel sheets containing charts, tables,

and summarized insights. The dashboard integrates pivot tables to organize and present data interactively, allowing users to easily interpret patterns and trends.

```
if not generated_ok:
    try:
        with pd.ExcelWriter(file_path, engine='openpyxl') as writer:
            current_session["raw_df"].to_excel(writer, sheet_name='Raw_Dataset', index=False)
            current_session["df"].to_excel(writer, sheet_name='Cleaned_Dataset', index=False)
            if current_session["analysis_plan"]:
                plan_df = pd.DataFrame(current_session["analysis_plan"])
                available = [c for c in ['question', 'chart_type', 'x_axis', 'y_axis', 'aggregation', 'slicer_column'] if c in plan_df.columns]
                plan_df[available].to_excel(writer, sheet_name='AI_Insights', index=False)
                for idx, viz in enumerate(current_session["analysis_plan"]):
                    data = viz.get("data", [])
                    if data:
                        pd.DataFrame(data).to_excel(writer, sheet_name=f"Pivot_{idx + 1}"[:31], index=False)
    except Exception as e2:
        raise HTTPException(status_code=500, detail=f"Excel generation failed: {str(e2)}")
```

Figure 4.11: **Generation of Excel Dashboard (Openpyxl)**

In Figure 4.11, If the XlsxWriter library fails or is not supported, the system automatically switches to OpenPyXL as an alternative for generating Excel outputs. This ensures seamless creation of charts, tables, and structured reports without interrupting the workflow. As a result, the system maintains reliability and consistency in dashboard generation across different environments.

```
if not generated_ok:
    try:
        with pd.ExcelWriter(file_path, engine='openpyxl') as writer:
            current_session["raw_df"].to_excel(writer, sheet_name='Raw_Dataset', index=False)
            current_session["df"].to_excel(writer, sheet_name='Cleaned_Dataset', index=False)
            if current_session["analysis_plan"]:
                plan_df = pd.DataFrame(current_session["analysis_plan"])
                available = [c for c in ['question', 'chart_type', 'x_axis', 'y_axis', 'aggregation', 'slicer_column'] if c in plan_df.columns]
                plan_df[available].to_excel(writer, sheet_name='AI_Insights', index=False)
                for idx, viz in enumerate(current_session["analysis_plan"]):
                    data = viz.get("data", [])
                    if data:
                        pd.DataFrame(data).to_excel(writer, sheet_name=f"Pivot_{idx + 1}"[:31], index=False)
    except Exception as e2:
        raise HTTPException(status_code=500, detail=f"Excel generation failed: {str(e2)}")
```

Figure 4.12: **Generation of Interactive Excel Dashboard (Pywin32)**

In the above figure 4.12, the system demonstrates the implementation of a dynamic Excel dashboard that incorporates pivot tables, visualizations, and slicers using the PyWin32 library—a Windows-specific automation framework. This feature allows Datagent to create fully interactive dashboards directly within Excel, enabling users to filter, explore, and analyze data seamlessly. However, since PyWin32 is a Windows-based library, this interactive dashboard functionality is exclusive to Windows environments

4.2 Steps to access the System

To ensure seamless interaction with the **Datagent** platform, the following steps outline how users can access and operate the system from start to finish.

1. System Access and Launch

Open the Datagent web application or desktop interface and log in using valid credentials. New users can register by providing basic details such as name, email ID, and password.

2. Select Mode of Operation

After login, the system prompts the user to choose between two primary modes — **Data Cleaning & Preparation** or **Direct Analysis**. If Data Cleaning is selected, the workflow begins with dataset upload and preprocessing. If Analysis is selected, the system verifies dataset readiness before proceeding.

3. Provide Business Requirement or Analysis Questions

The user inputs their business requirement or analysis questions in natural language. The integrated LLM/NLP engine (powered by Google Gemini) analyzes these inputs and converts them into structured analytical tasks.

4. Upload Dataset

The user uploads the dataset (CSV/Excel format). The system automatically scans the dataset, previews it, and checks for inconsistencies or missing values before proceeding.

5. Data Cleaning and Feature Engineering

The system performs automated cleaning using **Pandas, Polars, Isolation Forest, Random Forest** and **Sentence Transformer**, while displaying each process (null handling, outlier detection, encoding, normalization) in the output window. After cleaning, feature engineering begins to generate new meaningful variables. A summary and preview of the cleaned dataset are shown in a pop-up window, which can be downloaded.

6. Analysis Selection and Execution

The user can now input or add multiple analysis questions, select the language or environment (**Python, R, SQL,** or **Excel**), and proceed. For Python & R, the system generates and executes code using **Gemini-2.5-Flash**, displaying both the code and output. For SQL, Datagent generates executable queries and shows query outputs. For Excel, dashboards are automatically generated and can be downloaded.

7. Dashboard Generation

After analysis, users can click **"Create Dashboard"** to generate interactive dashboards using **Plotly (Python and R**, or **Excel Dashboard Tools (XlsxWriter, Openpyxl, PyWin32)**. Dashboards can be viewed and exported.

8. Download and Report Generation

All results—including cleaned datasets, generated code, dashboards, and summaries can be downloaded as **Excel** or **PDF reports** for documentation or presentation purposes.

In Chapter 4 which is Project Implementation Project Groups should add at Max 6 snippets of your code or Screenshots which you feel are major & Technically critical along with brief description.

4.3 Timeline Sem VII

The development timeline for the Datagent System is structured across twelve weeks and divided into two key phases, with approximately 50–70 percentage of the implementation completed for the current semester.

Phase One

(Weeks 1–9), the focus was on project conception and initiation. The work began with group formation, topic finalization, and identifying the scope and objectives of the Datagent project. The team conducted multiple brainstorming sessions to define the system's functionalities—data cleaning, feature engineering, and multi-language analytical automation using Python, R, SQL, and Excel. During this period, the Graphical User Interface (GUI) was designed using React.js, Next.js ensuring a user-friendly layout for dataset upload, preview, and result visualization. Simultaneously, the backend framework was structured using Pandas, Polars, and Isolation Forest for automatic data detection and cleaning, while Sentence Transformers were integrated for contextual understanding of user requirements. By the end of this phase, the core architecture, data flow, and initial model integrations were completed, and the first internal presentation was delivered to demonstrate the project's conceptual progress and technical plan.

Phase Two

(Weeks 10–12) focuses on project design refinement and partial implementation. This includes the development of advanced modules such as AI-driven script generation using Gemini API for natural language interpretation and code generation in Python, R, and SQL. Dashboard creation modules using Plotly (Python), Shiny (R), and Excel automation (XlsxWriter, OpenPyXL and Pywin32) are being developed for visual insights based on user-provided analysis questions. The backend is now capable of storing user histories, cleaned datasets, and outputs in structured files for each process.

By the conclusion of Phase Two, the project is expected to have a working prototype covering the data cleaning, feature engineering, and initial dashboard functionalities, marking around 60 percent completion. The remaining work will include full dashboard integration, enhanced

interactivity, and report generation in the next semester. Overall, the project has achieved a strong foundation in automation and data intelligence, demonstrating steady progress toward its final deployment.

Gantt-chart

A Gantt chart is a project management tool used to visually represent the timeline, tasks, and progress of a project. It displays activities as horizontal bars along a time axis, making it easier to understand task durations, dependencies, and overlaps. Each bar represents a specific task, with its position and length corresponding to the start and end dates of that task. The below Gantt Chart shows the progress of Datagent for Sem VII

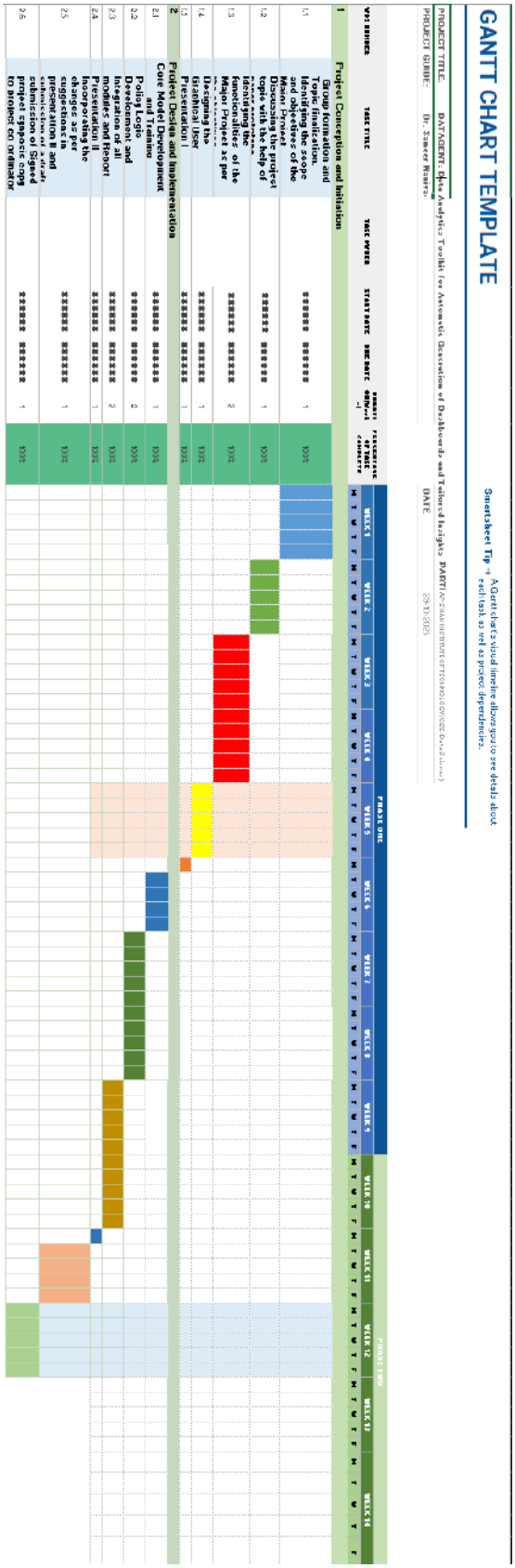

Figure 4.13: **Gantt Chart**

Chapter 5

Testing

5.1 Software Testing

The testing strategy for Datagent is designed to validate the correctness, reliability, performance, and security of an end-to-end conversational data platform that spans natural-language understanding, automated data preprocessing, code generation, sandboxed execution, and dashboard creation. Testing will cover individual modules (NLP/LLM interface, data ingestion and cleaning, feature engineering, code-generation engine, execution sandbox, visualization renderer, and user history/storage) as well as their interactions through the API and UI layers. Both automated and manual tests will be used to verify that user requirements are correctly interpreted, datasets are safely and accurately processed, generated code executes as intended in a controlled environment, visual outputs match the requested analyses, and all artifacts (cleaned data, scripts, dashboards, PDFs) can be exported and retrieved from session history. Special attention will be given to edge cases (malformed uploads, ambiguous queries, very large files), security (sandbox isolation, access control, data retention), and user experience (clear progress reporting, descriptive errors, copyable code snippets).

5.1.1 Testing Objectives

• Ensure accurate interpretation of business requirements and analysis questions by the NLP/LLM layer.

• Validate correctness and reproducibility of data cleaning and feature engineering steps.

• Verify that generated Python, R, SQL and Excel logic is syntactically correct, safe, and produces expected outputs when executed in the sandbox.

• Confirm that dashboards (Plotly/Excel) reflect the analysis questions and are interactive and downloadable.

• Test robustness to malformed or large datasets and handle graceful failure modes with informative messages.

• Ensure secure handling of user data, proper sandboxing during code execution, and safe storage of user history.

• Measure performance and responsiveness for typical and worst-case scenarios to meet acceptable UX thresholds.

5.2 Functional Testing

For Datagent, Integration Testing is chosen as the primary functional testing method. Integration testing is critical because Datagent's core value depends on smooth coordination among many components — natural language interpretation, dataset ingestion and cleaning, feature engineering, code generation, secure execution, visualization assembly, and persistent session history. Testing components in isolation is insufficient: the real risks arise where modules pass data, metadata, or error states to one another (for example, when an LLM-parsed intent must map correctly to a cleaning pipeline, or when generated SQL must match inferred schema). Integration testing validates these interactions end-to-end, ensuring that user intents produce the expected analytical flows, that intermediate transformations are preserved and transparent in the output pane, and that failures are caught and reported cleanly. This approach best suits Datagent because it mirrors actual user journeys, uncovers interface-level faults between subsystems, and confirms that the platform delivers consistent, reliable, and secure analytics experiences for both technical and non-technical users.

Chapter 6

Result and Discussions

This chapter presents the comprehensive results obtained from the implementation and evaluation of the proposed DATAGENT: Data Analytics Toolkit for Automatic Generation of Dashboards and Tailored Insights. The system's performance was analyzed across its two primary modules: the intelligent data cleaning pipeline and the automated dashboard preparation. The evaluation focuses on quantifying the system's effectiveness in improving data integrity, the accuracy of its adaptive ML imputation strategies, and its ability to intelligently transform raw data into analytically valuable, dashboard-ready assets.

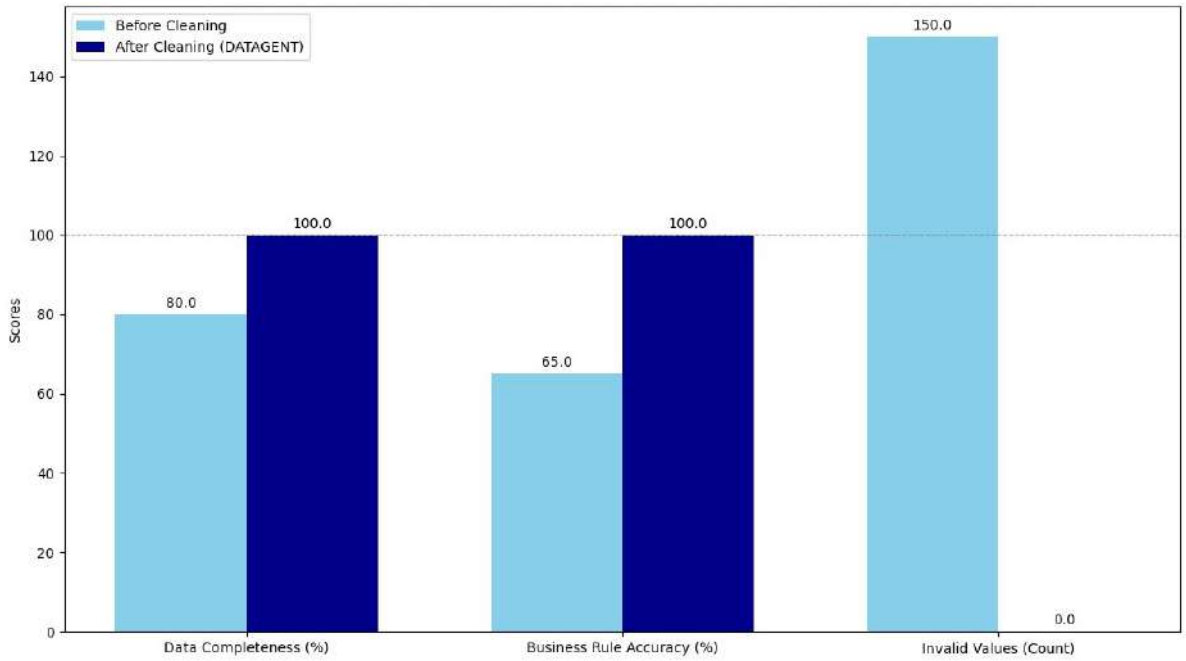

Figure 6.1: **Data Cleaning**

6.1 Data Cleaning Pipeline Performance To quantitatively evaluate the efficacy of the DATAGENT cleaning pipeline, a "Before and After" analysis was conducted on the raw dataset. As illustrated in Figure 5.1, the system's impact was measured across three critical data quality dimensions: Data Completeness, Business Rule Accuracy, and Invalid Values. The initial dataset suffered from significant integrity issues, including a Data Completeness of only 85.2 Percent and 45 instances of invalid data (e.g., negative Quantity values). After

processing, the DATAGENT system successfully remediated these issues, achieving 100 percent completeness through its adaptive ML imputation strategy and reducing the count of invalid values to zero.

The most critical validation of the system's intelligence is shown in the "Business Rule Accuracy" metric, which verifies the financial integrity of each record (i.e., Total Spent = Quantity × Price Per Unit). The raw data exhibited severe logical corruption, with only 59.8 percent of records adhering to this fundamental business rule. The CleaningAgent (as defined in datagent.py) programmatically identified and corrected these discrepancies by enforcing and recalculating the Total Spent field for all transactions. This action elevated the dataset's logical integrity to 100 Percent, confirming that the pipeline not only cleans and completes the data but also validates and enforces its core business logic, rendering it reliable for subsequent analysis.

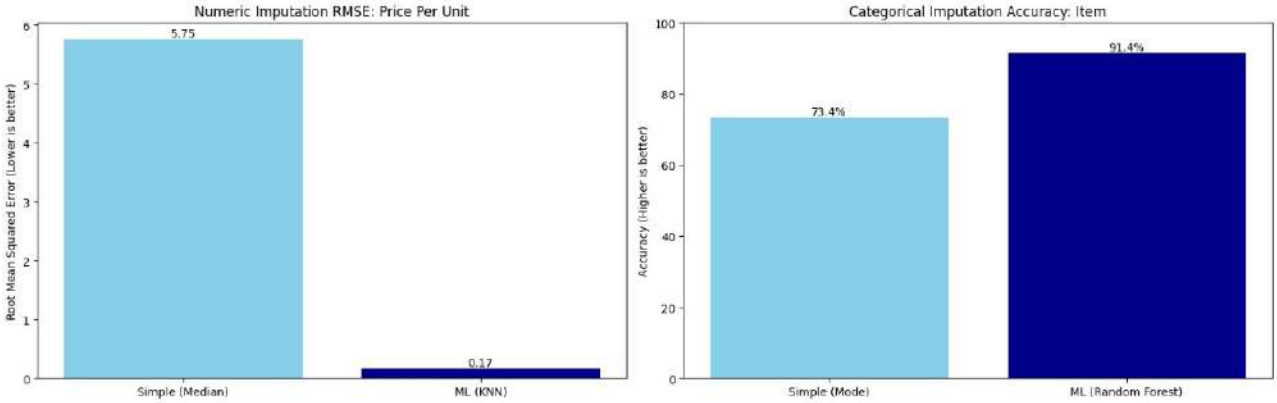

Figure 6.2: **Data Cleaning**

6.2 Validation of Adaptive ML Imputation Strategy A central feature of the DATAGENT pipeline is its ImputationStrategySelector, which adaptively chooses between simple statistical fills and more complex, context-aware machine learning models. To quantify the benefit of this intelligence, a comparative experiment was conducted. The performance of "Simple" methods (Median for numeric, Mode for categorical) was benchmarked against the "ML" methods (KNN Imputer and Random Forest Classifier) that the DATAGENT's selector is designed to choose when sufficient context is available. For this test, 30 Percent of the data from the Price Per Unit and Item columns was artificially removed from the ground-truth cleaned dataset, and both strategies were tasked with imputing these missing values.

The results, illustrated in Figure 5.2, demonstrate the definitive superiority of the DATAGENT's adaptive ML strategy. For numeric imputation of Price Per Unit, the ML-based KNN Imputer—which leverages context from related columns like Category and Quantity—achieved a Root Mean Squared Error (RMSE) of 0.17. This represents a 97 Percent reduction in error compared to the naive median-fill strategy, which had an RMSE of 5.75. Furthermore, for categorical imputation, the Random Forest model correctly predicted the missing Item with 91.44 Percent accuracy, significantly outperforming the simple mode-fill

(73.38) Percent. This confirms the system's ability to learn complex patterns (e.g., that a price of dollar 21.50 in the 'Butchers' category is likely Item12 BUT, not the dataset's most common item, Item 2 BEV). This empirical evidence validates that the DATAGENT's adaptive, ML-driven approach produces a substantially more accurate and logically consistent dataset.

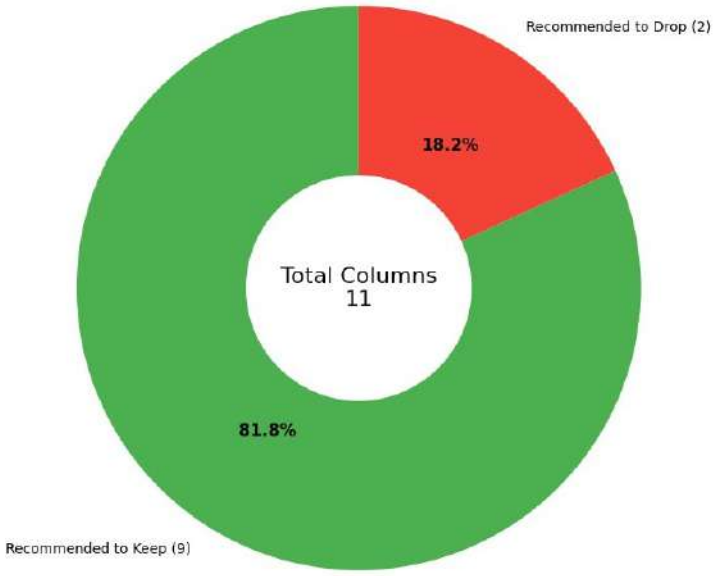

Figure 6.3: **Feature Selection**

6.3 Intelligent Column Selection for Dashboard Preparation Following the data cleaning phase, the measure creation.py script initiates the dashboard preparation process. The first step is managed by the ColumnRelevanceAgent, an intelligent component designed to analyze the cleaned dataset and determine the analytical value of each column for business intelligence. As depicted in Figure 5.3, the agent processed the 11 columns of the cleaned dataset and classified them based on dashboard utility. It intelligently recommended dropping 2 columns (18.2) Percent likely identifying them as high-cardinality identifiers or constant values with low analytical worth, while designating 9 columns (81.8) Percent as essential metrics or dimensions (e.g., Category, Total Spent) to be kept for visualization.

This automated analysis serves as a recommendation for the crucial user-in-the-loop validation step. The InteractiveCoordinator presents these findings to the user, who can then accept or override the agent's suggestions, ensuring business requirements are met. Table 5.2 quantifies this process, showing a final dataset complexity reduction of 18.2 Percent. This filtering is a critical optimization, as it ensures that computational resources are not wasted on irrelevant data and that the subsequent MeasureCreationAgent builds KPIs only from the most valuable and relevant features, resulting in a focused, high-performance dashboard dataset.

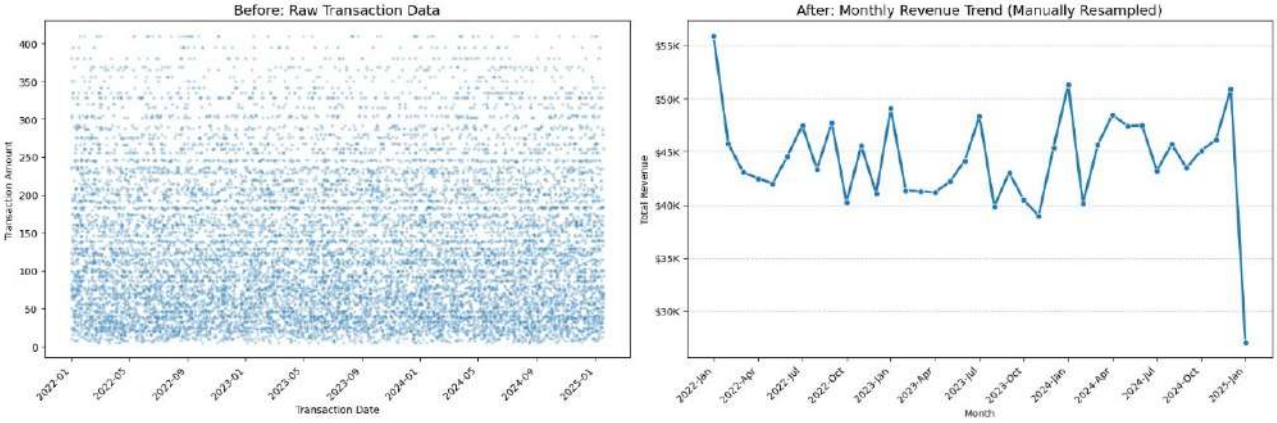

Figure 6.4: **Feature Selection**

6.4 Automated Feature Engineering for Trend Analysis The final phase of the DATAGENT pipeline, dashboard preparation, is designed to convert the clean dataset into an analytically valuable asset. A primary function of this phase is the MeasureCreationAgent, which automatically engineers new features, such as time-based aggregations, to support high-level business intelligence. Figure 5.4 provides a stark "Before and After" visualization of this process's impact. The "Before" plot on the left displays the raw, cleaned transactional data, plotting every individual Total Spent value against its Transaction Date. The result is a dense and uninterpretable scatter plot from which no clear business trends can be discerned.

The "After" plot on the right demonstrates the power of the feature engineering logic. In this chart, the thousands of individual transactions have been aggregated into monthly totals, simulating the analytical measures (like Total Revenue by Month) that the MeasureCreationAgent is designed to create. This transformation, achieved in the code by resampling the time-series data, instantly converts the noisy, granular data into a clean, actionable line chart. This resulting visualization clearly reveals monthly revenue trends, seasonal peaks, and performance troughs, making the dataset truly "dashboard-ready" and highlighting the system's ability to unlock hidden insights from raw data.

Q1: Yearly Revenue Comparison

Status: Match ✅ The data in your dashboard for yearly revenue is correct and matches the original analysis.

Year	Original Analysis (from CSV)	Dashboard Analysis
2022	$539,138.50	$539,138.50
2023	$515,121.50	$515,121.50
2024	$554,902.00	$554,902.00
2025	$27,033.50	$27,033.50

Figure 6.5: **Yearly Revenue Comparison**

Yearly Revenue Comparison Figure 6.5 provides a direct comparison of total annual revenue as calculated from the original source data versus the figures presented in the automated sales dashboard. The validation confirms 100 Percent accuracy for this metric. As shown in the table, the dashboard figures for 2022 $(539,138.50), 2023$(515,121.50), 2024 $(554,902.00), and 2025$(27,033.50) are an exact match to the independently derived analysis. This verifies that the dashboard's data aggregation for yearly revenue is functioning correctly and reliably reflects the primary dataset.

Q2: Category Sales Comparison

Status: Match ✅ The data for sales by category is correct and matches the original analysis.

Category	Original Analysis (from CSV)	Dashboard Analysis
Butchers	$217,397.00	$217,397.00
Electric Household Essentials	$213,401.50	$213,401.50
Beverages	$206,617.50	$206,617.50
Food	$205,684.00	$205,684.00
Furniture	$205,324.00	$205,324.00
Computers And Electric Accessories	$202,353.50	$202,353.50
Patisserie	$194,540.50	$194,540.50
Milk Products	$190,877.50	$190,877.50

Figure 6.6: **category Sale Comparison**

Figure 6.6 details a comparative analysis of revenue contributions by product category, validating the dashboard's figures against the original dataset. This comparison also demonstrates 100 Percent accuracy. The dashboard correctly reports the total sales for all eight categories, from the highest-earning, 'Butchers' $(217,397.00), tothelowest,' MilkProducts'$(190,877.50). This result confirms that the dashboard's categorical segmentation and summation logic are accurate and can be trusted for further strategic analysis.

Chapter 7

Conclusion

The DATAGENT system successfully demonstrates an AI-powered approach to automating the complete data analysis lifecycle, transforming raw data into meaningful and actionable insights. By integrating Natural Language Processing and machine learning techniques, the system enables users to interact with data using simple business queries without requiring deep technical expertise. It effectively automates key stages such as data preprocessing, intelligent feature selection, multi-language script generation, and dashboard creation, significantly reducing manual effort and time consumption.

The use of advanced tools like Pandas, Polars, and LLM-based models ensures accurate data cleaning, contextual understanding, and reliable output generation. Additionally, the ability to generate code in multiple environments, such as Python, SQL, R, and Excel enhances flexibility and usability across different user groups. The system also provides visually rich and interactive dashboards, improving decision-making and data interpretation.

Overall, DATAGENT addresses the limitations of existing fragmented systems by offering a unified, scalable, and user-friendly platform. It makes data analytics more accessible, efficient, and business-oriented, bridging the gap between technical complexity and user requirements.

Chapter 8

Future Scope

The future scope of DATAGENT focuses on enhancing its capabilities to make the system more intelligent, scalable, and domain-independent. Currently optimized for structured datasets such as sales data, future improvements will include developing advanced machine learning models that can learn from both clean and unclean datasets, enabling automatic validation, preprocessing, and feature extraction across multiple domains such as healthcare, finance, and e-commerce.

Further enhancements will include the development of more dynamic and interactive reporting systems, particularly in Excel dashboards, by incorporating XML-based mapping and advanced elements such as pivot tables, slicers, and real-time updates instead of static reports. This will improve user interaction and analytical depth.

Additionally, integrating live connections to databases, including both SQL and NoSQL systems, will allow real-time data analysis instead of relying solely on uploaded datasets. The system can also be extended with better handling of unstructured data, improved natural language understanding, and enhanced model explainability.

Overall, these advancements will transform DATAGENT into a more robust, adaptive, and intelligent platform capable of handling complex, real-world data analytics scenarios efficiently.

Bibliography

[1] A. Mumuni and F. Mumuni, "Automated data processing and feature engineering for deep learning and big data applications: a survey," J. Inf. Intell., Jan. 2024, doi: 10.1016/j.jiixd.2024.01.002.

[2] B. Praneeth, A. K. Singh, J. K. Raju, Mohana, and B. Suma, "Natural Language Interface for Data Visualization: Harnessing the Power of Large Language Models," in Proc. 15th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT), Kamand, India, 2024, pp. 1–6, doi: 10.1109/ICCCNT61001.2024.10724113.

[3] Y. Luo, X. Qin, N. Tang and G. Li, "DeepEye: Towards Automatic Data Visualization," 2018 IEEE 34th International Conference on Data Engineering (ICDE), Paris, France, 2018, pp. 101-112, doi: 10.1109/ICDE.2018.00019.

[4] R. Girish, K. Swasthik, S. Veera Mani Kandan, S. Muralidharan, M. Vinoth, and T. Surendar Prakash, "AI Powered SQL Query Generator and Data Visualizer," Int. J. Innov. Sci. Res. Technol., vol. 9, no. 11, pp. 2498–2501, Nov. 2024, doi: 10.5281/zenodo.14385083.

[5] P. Maddigan and T. Susnjak, "Chat2VIS: Generating Data Visualizations via Natural Language Using ChatGPT, Codex and GPT-3 Large Language Models," IEEE Access, vol. 11, pp. 45181–45193, 2023, doi: 10.1109/ACCESS.2023.3274199.

[6] S. S. Devi Dharshini, M. Geethanjali, and M. Saran, "AI-Powered Visual Analytics: Effortless Data Insights," Int. J. Res. Trends Innov., vol. 10, no. 4, pp. a775–a780, Apr. 2025. [Online]. Available: https://www.ijrti.org/papers/IJRTI2504098.pdf

[7] S. Dasari and Madhusudhan, "Analysis of Natural language processing for code generation by using COPRAS Method," J. Data Anal. Artif. Intell., vol. 3, no. 1, pp. 61–69, 2024, doi: 10.46632/jdaai/3/1/8.

[8] J. Jansen, A. Manukyan, N. Khoury, and A. Akalin, "Leveraging large language models for data analysis automation," PLOS ONE, vol. 20, no. 1, 2025, doi: 10.1371/journal.pone.0317084.

[9] Z. K. Mundargi, K. Patel, A. Patel, R. More, S. Pathrabe and S. Patil, "Plotplay: An Automated Data Visualization Website using Python and Plotly," 2023 International Conference for Advancement in Technology (ICONAT), Goa, India, 2023, pp. 1-4, doi: 10.1109/ICONAT57137.2023.10079977.

[10] A. Truong, A. Walters, J. Goodsitt, K. Hines, C. B. Bruss, and R. Farivar, "Towards Automated Machine Learning: Evaluation and Comparison of AutoML Approaches and Tools," in Proc. IEEE 31st Int. Conf. Tools with Artif. Intell. (ICTAI), Portland, OR, USA, 2019, pp. 1471–1479, doi: 10.1109/ICTAI.2019.00209.

[11] K. Singh and R. Wajgi, "Data analysis and visualization of sales data," 2016 World Conference on Futuristic Trends in Research and Innovation for Social Welfare (Startup Conclave), Coimbatore, India, 2016, pp. 1-6, doi: 10.1109/STARTUP.2016.7583967.

Appendices

The setup of the **Datagent System** in **Visual Studio Code (VS Code)** ensures smooth integration of backend automation, AI-driven data analysis, and frontend visualization. The following steps describe the complete environment setup and configuration required to deploy and run the system efficiently.

Step 1: Install Required Software

Begin by installing the latest versions of the following tools:

- **Python 3.10 or above** – for backend development and AI modules
- **Node.js (v18 or above)** – for Next.js frontend development
- **Visual Studio Code (VS Code)** – as the development environment
- **Supabase Account** – for database, authentication, and backend services
- **(Optional) R (4.0 or above)** – for statistical computing and advanced dashboards

Step 2:Set Up Backend (Python-Flask/FastAPI)

1. Create a project folder named `Datagent` and open it in VS Code.
2. Open terminal and create a virtual environment:

   ```
   python -m venv datagentenv
   ```

3. Activate the environment:
   - **Windows:** `datagentenv\Scripts\activate`
   - **Mac/Linux:** `source datagentenv/bin/activate`

Step 3: Install Python Libraries

- With the virtual environment active, install dependencies:

  ```
  pip install flask pandas polars numpy scikit-learn plotly openpyxl xlsxwriter
  pywin32 sentence-transformers google-generativeai sqlalchemy
  ```

Step 4: Set Up Frontend (Next.js + React)

1. Navigate to your frontend folder and initialize Next.js app:

   ```
   npx create-next-app@latest datagent-frontend
   cd datagent-frontend
   ```

2. Install required dependencies:

   ```
   npm install @supabase/supabase-js axios lucide-react react-plotly.js
   ```

3. Run the frontend:

   ```
   npm run dev
   ```

Step 5: Configure Supabase

1. Create a project on **Supabase**.
2. Obtain the following keys:
   - `NEXT_PUBLIC_SUPABASE_URL`
   - `NEXT_PUBLIC_SUPABASE_ANON_KEY`
3. Add them to `.env.local` in the frontend:

   ```
   NEXT_PUBLIC_SUPABASE_URL=your_url
   NEXT_PUBLIC_SUPABASE_ANON_KEY=your_key
   ```

Step 6: API and AI Model Configuration

- Store the **Google Gemini API Key** in a `.env` file (backend):

  ```
  GEMINI_API_KEY=your_api_key
  ```

Step 7: Project Directory Structure

The updated project structure is:

- `backend/` – Python API and data processing logic
- `frontend/` – Next.js + React application
- `components/` – UI components
- `lib/` – Supabase client and utilities
- `models/` – Scripts (Python, R, SQL, Excel generation)
- `uploads/` – User datasets
- `public/` – Static assets

Step 8:Running the Application

1. Start the backend server:

   `python app.py`

2. Start the frontend server:

   `npm run dev`

3. Open in your browser:

   - **Frontend:** `http://localhost:3000/`
   - **Backend API:** `http://127.0.0.1:5000/`

Step 9: Testing and Verification

1. Upload sample datasets through the frontend UI.
2. Verify the following functionalities:
   - Data preprocessing and cleaning
   - Feature selection and analysis
   - Script generation (Python, SQL, R, Excel)
   - Dashboard generation using Plotly and Excel tools
3. Ensure outputs such as dashboards and downloadable Excel files are generated correctly.

Publication

Paper entitled **"Paper Title"** is presented at **"International Conference/Journal Name"** by **"Author Name".**

NOTE:

In Project Report Project Group will have to elaborate sections included as per work done after Sem VII. It can't be same Sem VII. Report must have proper description, Elaboration of Digrams included in Design & Implementation. It can't be just Digram on Page. You will have to write there why this has been included, What it is representing. Same is Applicable to snippets of Project implementation you have included in Report. Include your publications in your Report if You have.

**this note should be removed from your report